

Natural Language Processing

Support Ticket Classification

1. INTRODUCTION

The goal of this project is to classify user IT support queries into one of 14 different support ticket types. The dataset consists of two columns: the first contains the text of the support request and the second contains the manually assigned class for each request. The task is especially difficult because the text for the support request is very noisy and omits a lot of context which a machine learning model might not be able to infer unlike a human. Users also write in a short form avoiding complete sentences, use abbreviations like “acct” and “pwd” and misspell some words like “emial”. Additionally, there are just 308 labelled samples and varying amounts of data for each class: the largest class is payment_issue (33 samples) and the smallest is two_factor_auth (10 samples). Also, some classes are semantically similar (such as shipping_delay and order_tracking) making classification into one category very difficult. A successful classifier and preprocessing pipeline should be able to address these issues.

2. DATA HANDLING

The two pipelines vary only by preprocessing, both classifying support requests into classes using logistic regression. This ensures that any differences in performances are only due to different preprocessing. The different preprocessing pipelines are summarised in Figure 1 below:

Stage	Pipeline 1	Pipeline 2
Shared cleaning	Lowercasing; domain-based spelling and abbreviation normalisation; punctuation, hidden character and whitespace removal; tokenisation; followed by stopword removal.	Same as Pipeline 1.
Reduction step	Stemming.	Lemmatisation using different POS tags for different word types.
Vectorisation	TF-IDF over unigrams.	Bag-of-words counts over bigrams.
Expected behaviour	Less sparse; highlights informative unigrams.	Captures key phrases; more brittle to unseen wording.

Figure 1. Preprocessing Pipeline Comparison

Both pipelines share a domain specific spelling and abbreviation normalisation step where common IT words are corrected and normalised to a canonical form like “acct” to “account”, “pwd” to “password” and variations of two factor

authentication to “2fa” using regular expression. Common non-IT words are also corrected like “pls” to “please” and “w/” to “with”. This ensures that all variations of the same term are treated as one feature improving accuracy. “2” was also changed to “to”, though this may create issues when “2” is used in the context of numbers. Stop word removal was performed in both pipelines to reduce sparsity and remove noise improving performance and computational load. Stop word and punctuation removal is safe for text classification as it is unlikely to remove important context and meaning.

Pipeline 1 uses Bag of Words with TF-IDF to reduce noise from frequent, uninformative words which appear across all classes. Both methods typically work well for text classification. TF-IDF should help with BoW’s sensitivity to common tokens and improve the relevance of rare words. Additionally, scikitlearn’s implementation of TF-IDF uses smoothing to prevent division by zero (by adding a 1 to the numerator and denominator of IDF and to the full log term) (Scikit learn, 2018). However, BoW loses order and context whilst TF-IDF ignores semantics. Additionally, stemming is used which also reduces the context. When constructing this model, I conjectured that ignoring context and semantics would not negatively impact results in text classification.

Pipeline 2 uses Bag of 2-Grams to try and capture context through the use of bigrams like reset password and update email (see Figure 2) and might be able to capture more domain specific expressions like credit card. Bo2G also works well for the short documents here. However, bigrams results in larger vocabulary size (303 in Pipeline 2 vs 284 in Pipeline 1) and less computational efficiency. Lemmatisation is also used which maintains context and is more interpretable than stemming (see Figure 2) but is slower. I thought that this pipeline might be better at distinguishing between semantically similar classes by better capturing meaning and context. However, there is a big risk in that using only bigrams make the representation much sparser: this is very sensitive to unseen phrases and alternative wording (very likely given limited number of samples and short queries).

A few examples of differences in tokenisation are presented below (copy vs copi, delete vs delet, permanently vs perman: lemmatisation vs stemming) (see Figure 2).

text	Pipeline 1 tokenised	Pipeline 2 tokenised	label
------	----------------------	----------------------	-------

download GDPR data copy - steps?	[download, gdpr, data, copy, step]	[download, gdpr, data, copi, step]	data_privacy
how 2 delete my acct permanently?	[delete, account, permanently]	[delet, account, perman]	delete_account
cant update emial if acct is locked wat do?	[update, email, account, lock]	[updat, email, account, lock]	update_email

Figure 2. Differences in Tokenisation

3. MODELLING

Both pipelines use logistic regression as the classifier. This was used instead of more sophisticated methods like neural networks because of ease of implementation and efficiency to train. However, performance is hindered since it assumes linearly separable data – neural networks should be able to express non-linear, complex patterns better and might perform better if this data is not close to linearly separable. But given the data is limited, neural networks are more prone to overfitting than logistic regression. Lasso or Ridge regularisation could also be used to reduce overfitting and improve performance. Logistic regression is also highly interpretable through feature weights reflecting feature importance which is important in the context of IT support (Geeksforgeeks.org, 2026). For example, this interpretability has been used to inspect the highest weighted, most important n grams for each class (for both pipelines) to ensure that the model is learning relevant n grams for each class and not just noise.

4. EVALUATION

Both pipelines use a 80/20 train/test split (stratified to ensure that each class is proportionally represented in both splits) and are also evaluated using 5-fold stratified cross validation for a more reliable estimate of model performance. Macro-averaged precision, recall and F1 are used to evaluate performance to ensure that the models perform well on all classes, not just the largest classes (Geeksforgeeks, 2025).

Pipeline 1 has much stronger performance than Pipeline 2 in both held-out test set and on averaging over 5 folds using cross validation (see Figures 3 and 4): higher accuracy, macro-averaged recall, precision and F1 scores (a good measure of performance for imbalanced datasets, by combining precision and recall) and lower standard deviations across the board.

Metric	P1 Test	P2 Test	P1 5-fold CV	P2 5-fold CV
Accuracy	0.9355	0.6935	0.8866 ± 0.0349	0.5911 ± 0.0953
Macro precision	0.9506	0.7929	0.9001 ± 0.0414	0.7215 ± 0.0485
Macro recall	0.9167	0.6583	0.8688 ± 0.0415	0.5623 ± 0.0926
Macro F1	0.9222	0.6766	0.8693 ± 0.0436	0.5805 ± 0.0893

Figure 3. Performance Comparison

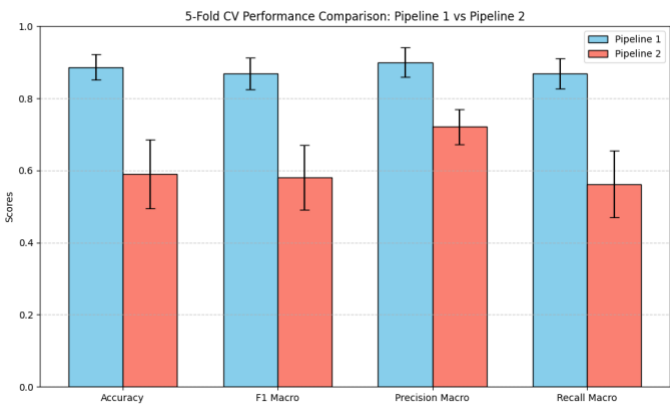


Figure 4. Performance Comparison

The majority class predictor gives just 10.7% accuracy so clearly both models are learning meaningful structure but Pipeline 1 is substantially more reliable and balanced across each class: Pipeline 1 makes at most 1 error for any given true label, whereas Pipeline 2 has as many as 3 errors for some true labels like `billing_history` (see Figure 5).

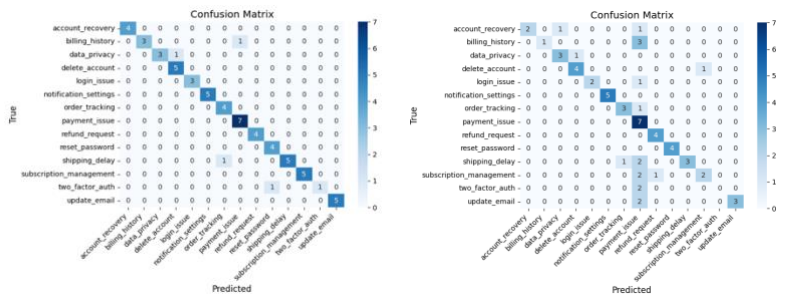


Figure 5. Confusion Matrices for Pipelines 1 and 2 under the 80/20 train/test split

Pipeline 1 performs better since the queries are short and noisy and so unigrams better carry information about the class. For example, in Pipeline 2 the tokenised text `[get, security, warn, every, time, try, login]` is misclassified as `payment_issue` instead of `login_issue` likely because the unigram “login” is more informative and common than the rarer bigram “try login”. In Pipeline 1 the same query is classified correctly. Additionally in Pipeline 1, the highest weighted unigrams learnt by the classifier are semantically convincing: ‘2fa’, ‘code’ and ‘devic’ for ‘two_factor_auth’ and ‘password’, ‘reset’ and ‘forgot’ for ‘reset_password’. This gives confidence that the model is learning useful predictive features and not random noise. In Pipeline 2, for some classes the bigrams seem very informative like ‘reset password’ for ‘reset_password’ or ‘cancel subscription’ for ‘subscription_management’, but less informative/convincing looking at ‘mandatory account’ and ‘2fa mandatory’ for ‘two_factor_auth’ (see Figure 6).

account_recovery	: ['recoveri', 'recov', 'account', 'help', 'email']	['recover account', 'account recovery', 'recovery fail', 'fail next', 'email possible']
billing_history	: ['bill', 'histori', 'invoic', 'past', 'download']	['bill history', 'download invoice', 'access past', 'past bill', 'duplicate charge']
data_privacy	: ['data', 'person', 'request', 'access', 'restrict']	['personal data', 'data request', 'data company', 'data share', '3rd party']
delete_account	: ['delet', 'account', 'remov', 'reactiv', 'step']	['delete account', 'account deletion', 'confirmation deletion', 'deletion immediate', 'deletion get']
login_issue	: ['login', 'log', 'cant', 'sign', 'unabl']	['log account', 'cant log', 'im unable', 'unable sign', 'sign account']
notification_settings	: ['notif', 'event', 'set', 'push', 'stop']	['notification setting', 'push notification', 'tone event', 'notification tone', 'event possible']
order_tracking	: ['track', 'order', 'mark', 'deliv', 'shipment']	['track order', 'track international', 'track information', 'international order', 'mark deliver']
payment_issue	: ['payment', 'card', 'work', 'bank', 'transfer']	['credit card', 'bank transfer', 'block account', 'payment block', 'work fail']
refund_request	: ['refund', 'request', 'long', 'get', 'method']	['refund request', 'request refund', 'get refund', 'limit number', 'reject explain']
reset_password	: ['password', 'reset', 'tablet', 'forgot', 'follow']	['reset password', 'step follow', 'follow new', 'reset step', 'password reset']
shipping_delay	: ['ship', 'deliveri', 'delay', 'order', 'packag']	['order delay', 'delivery time', 'package arrive', 'delay common', 'shipping delay']
subscription_management	: ['subscrip', 'plan', 'paus', 'cancel', 'temporarili']	['cancel subscription', 'pause subscription', 'change plan', 'subscription temporarily', 'temporarily eg']
two_factor_auth	: ['2fa', 'authent', 'sm', 'code', 'devic']	['mandatory account', '2fa mandatory', '2fa account', 'happens disable', 'enable 2fa']
update_email	: ['email', 'updat', 'new', 'chang', 'activ']	['update email', 'new email', 'change email', 'email account', 'email address']

Figure 6. Top 5 predictive features per class for Pipelines 1 and 2

Pipeline 1 only misclassifies 4/62 test samples with each error reflecting a close boundary. “How do I reset my two-factor authentication device?” becomes reset_password instead of two_factor_auth because ‘reset’ is a competing unigram, “shipment marked delivered but not received!” is predicted as order_tracking instead of shipping_delay because both contain similar delivery/tracking language like ‘deliv’/‘deliveri’.

Pipeline 2 is much weaker misclassifying 19/62 and repeatedly overpredicting payment_issue such as for “order tracking not working properly?” perhaps because this is the most common class label. Although this pipeline is able to capture more context through the use of lemmatisation and Bo2G, this does not translate to better performance due to unseen queries being phrased differently and so not matching the bigrams learnt during training. This is more pronounced for “two_factor_auth” which only has 10 samples total and 2 in the test set. This results in a model with a reasonably high precision but much worse recall, meaning it is confident when it predicts but often fails to fire for many real classes.

5. CONCLUSIONS

Overall, the pipelines demonstrate that careful preprocessing is just as important as model choice for short support ticket data. The best model is Pipeline 1: domain specific normalisation, stemming and TF-IDF unigrams. This pipeline is adapted to short, noisy queries by retaining informative words, reducing the effect of common, uninformative words and by reducing words to a canonical form. The model vastly outperforms Pipeline 2 because of this. Reflecting on the results, both models might have improved performance using a combined bigram and unigram approach (ngram_range=(1,2)) to incorporate more context and single word meanings, respectively. Although it did not affect results, greater care should be taken in changing ‘2’ to ‘to’ such as using a rule-based approach: when ‘2’ appears before a verb, change ‘2’ to ‘to’. I also experimented with pyspellchecker for improved spelling correction, though this had an issue with domain specific words such as ‘Gdpr’ -> ‘gear’. An IT trained spellchecker or using character n grams could address this. Additional limitations include a small dataset and class imbalances which could be addressed using synonym replacement or other data

augmentation techniques.

6. REFERENCES

GeeksforGeeks (2025). *Macro average vs Weighted average*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/machine-learning/macro-average-vs-weighted-average/>.

Geeksforgeeks.org. (2026). *GeeksforGeeks / 404*. [online] Available at: <https://www.geeksforgeeks.org/data-science/advantages-and-disadvantages-of-logistic-regression/https://www.geeksforgeeks.org> [Accessed 27 Mar. 2026].

Scikit learn (2018). *TfidfVectorizer*. [online] Scikit-learn.org. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.